

Agent as a Service (AaaS)

Reimagining Cloud Support and Transformation

Through

Agentic AI

A unified Google Cloud blueprint for autonomous, outcome-driven cloud delivery — built on Vertex AI, Gemini, and the Google Cloud platform

Prepared by Satya prakash Chinnam & Cognizant Transformation Team

Strategy & Enterprise Transformation

Unified Edition — Version 4.0 | 2026

Last updated on: Aug 21, 2026

CONFIDENTIAL — Prepared for internal Google Cloud strategic review

Google Cloud

Table of Contents

Executive Summary **4**
1. The Limits of Legacy Cloud Support 5
    1.1 Tiered Support Doesn't Match Need 5
    1.2 The Response-versus-Resolution Gap 5
    1.3 ITSM Friction and Context Loss 5
    1.4 Talent Scarcity and the Expertise Bottleneck 6
2. The AaaS Paradigm Shift 7
    2.1 From Support Tiers to a Service Bouquet 7
    2.2 From Reactive to Predictive Operations 8
    2.3 Minimal Human Intervention by Design 8
    2.4 The Hyperscaler Landscape : A Narrowing Window 9
3. The AaaS Agent Portfolio 10
    3.1 Migration & Adoption Agents 10
    3.2 Operations & Reliability Agents 11
    3.3 Security & Compliance Agents 11
    3.4 Financial & Advisory Agents 11
4. The Autonomous Workload Lifecycle 12
5. Industry-Agnostic Approach **14**
6. Go-to-Market & Expansion Plan **14**
    6.1 Technical Solidity 14
    6.2 Ease of Implementation 14
    6.3 Commercial Expansion 14
7. Financial Business Case **14**
8. Technical Architecture Blueprint 16
    8.1 Current Model vs. Proposed AaaS model (Agent Triad) 17
9. Implementation Roadmap 18
    9.1 Phased Build 18
    9.2 Adoption Maturity Model 19
10. AaaS PoC Architecture 21
11. Governance, Risk, and Responsible AI 22
12. Conclusion & Strategic Advantage 24
Appendix A: Agent Capability Quick Reference **24**
Appendix B: Glossary **25**

Abstract

This whitepaper introduces "Agent as a Service" (AaaS), a new model for Google Cloud support that augments traditional human-led, reactive ticketing with a portfolio of autonomous AI agents. Instead of relying on manual fixes for technical issues, these agents monitor environments and proactively resolve problems, significantly improving resolution speed and operational efficiency.

The model replaces rigid support tiers with a flexible, usage-based "capability bouquet". Customers can subscribe to specific agents—such as those for cloud migration, financial optimization, or security—which orchestrate their tasks through the open Agent2Agent (A2A) protocol.

AaaS uses a graduated maturity model, allowing for varying levels of autonomy based on risk and policy. While routine infrastructure remediation can be fully autonomous, higher-risk actions or regulated tasks require "Human-in-the-Loop" (HITL) intervention, ensuring humans retain oversight and policy control.

This approach aims to reduce cloud engagement costs and operational waste by shifting the human role from manual operators to policy-makers. By leveraging native Google Cloud telemetry and reasoning capabilities, AaaS provides continuous, goal-driven support that evolves with the customer's cloud maturity, moving away from simple response-time metrics toward faster, more reliable resolutions.

This whitepaper outlines the potential commercialization of specific customer/industry agents to be a monetization vector within Cloud Support and other services (TAM etc) in Cloud in 2027.

Executive Summary

The cloud industry has reached an inflection point. Enterprises are accelerating their cloud journeys, but the delivery model behind that journey — tiered support packages, reactive ticketing, and labor-intensive consulting — has not kept pace with the speed and complexity that modern, cloud-native transformation demands.

This white paper introduces Agent as a Service (AaaS): a Google Cloud-native delivery model in which a curated portfolio of purpose-built AI agents, built on Vertex AI and powered by Gemini, becomes the primary interface between Google Cloud and its customers. Legacy models measure success by how quickly a human responds to a failure. AaaS measures success by how effectively an autonomous agent prevents, resolves, or advances a customer's cloud maturity — with minimal human intervention and full alignment with Google's Responsible AI principles.

Rather than selecting a fixed support tier, customers assemble a personalized “capability bouquet” — a Migration Navigator, a FinOps Optimizer, a Security Sentinel, a Compliance Guardian — each priced by usage and orchestrated through the open Agent2Agent (A2A) protocol against the customer's live Google Cloud environment.

Key Value Proposition

1. Operational Efficiency & Cost Reduction: Instead of relying on engineers to manually fix routine technical issues, AI agents continuously monitor single-project telemetry and execute Level-1 remediations. Consequently, AaaS dramatically cuts operational expenditure and reduces total Cloud engagement costs.

2. Speed & Agility: AI agents don't just "respond" to a ticket; they start fixing the problem immediately. This approach cuts the time it takes to move to the cloud by **40–60%** and can lower the cost of fixing incidents by up to **70%**. You get instant technical progress instead of waiting for a support person to get assigned.

3. Better Business Value: This model moves away from basic hourly consulting and toward high-value, automated results. Profit margins grow from the usual **20–30%** for human-led support to **65–80%** for AI-driven services. Customers get a predictable cost structure that is directly tied to the actual results they achieve.

This document sets out the strategic rationale for AaaS, a representative agent portfolio, the operating model and workload lifecycle, the financial case for both Google Cloud and its customers, a technical architecture built on Google Cloud infrastructure, a phased implementation roadmap with a four-stage adoption maturity model, and the governance framework required to deploy it responsibly.

1. The Limits of Legacy Cloud Support

Cloud adoption was meant to be liberating — faster delivery, elastic scale, and operational excellence. For most enterprises, the lived experience is fragmented tooling, opaque support, and a vendor relationship that cannot keep pace with their transformation ambitions. The root cause is structural: the support model itself has not evolved to take advantage of what an agentic, Google Cloud-native approach makes possible today.

1.1 Tiered Support Doesn't Match Need

Hyperscaler and managed-service support is organized into subscription tiers — typically Developer, Business, and Enterprise/Premium — priced primarily by account revenue rather than workload risk. A startup running a mission-critical payment workload may sit on Business-tier support, while a large enterprise with low-criticality dev/test environments over-purchases Premium coverage. Tier selection is investment-correlated, not need-correlated, so customers routinely pay for coverage that does not match their actual risk topology.

1.2 The Response-versus-Resolution Gap

Standard support agreements emphasize response-time Service Level Agreements — an acknowledgment within 15 minutes for a critical failure — which creates a strategic blind spot. A fast response usually means a human dispatcher posting a placeholder update to meet response SLOs, which introduces 'placeholder fatigue' and noise for the customer while the underlying system remains degraded. By contrast, an autonomous agentic approach immediately begins active, telemetry-driven investigation, providing substantive technical progress instead of generic updates.

1.3 ITSM Friction and Context Loss

The Service Management portals — Salesforce, ServiceNow, Jira Service Management, Zendesk, or an equivalent enterprise system — is the primary customer touchpoint for support, and it introduces friction at every step:

- **Manual ticket creation:** customers navigate category trees and mandatory fields, then wait for an engineer to independently rediscover what the customer already knows about their own environment.
- **Context loss at every handoff:** as a ticket moves from Tier 1 triage to Tier 2 generalists to Tier 3 domain specialists, context must be re-established at each boundary, consuming time and creating room for misdiagnosis.

- **Institutional memory loss:** without a persistent context fabric, customers are repeatedly burdened with re-explaining architectural nuances and compliance guardrails to a new engineer each time.
- **Static self-service:** FAQ articles and runbooks cannot adapt to a customer's specific environment or failure mode, so most customers abandon self-service after one or two failed attempts and file a ticket anyway.
- **Unresolved Closures:** Traditional support models often close tickets under 'unresponsive customer' protocols after a workaround is proposed, leaving the team unsure if the technical issue was resolved. AaaS eliminates this by using real-time telemetry to autonomously validate that system health has fully recovered before closing the loop.

1.4 Talent Scarcity and the Expertise Bottleneck

Cloud engineering expertise is among the scarcest commodities in the technology labor market. Specialized skills — GKE networking, Spanner performance tuning, multi-cloud security architecture — concentrate on a small number of senior engineers who become the constraint for every complex escalation. High turnover repeatedly erodes institutional knowledge of customer environments, and time-zone coverage gaps create dead zones that are especially costly for global enterprises, since production incidents do not respect business hours.

Pain Domain	Symptom	Business Impact
Rigid support tiers	One-size-fits-all packages priced by revenue, not risk	Overpayment or coverage gaps
Response vs. resolution	SLA “acknowledged” ≠ issue fixed	Extended MTTR, revenue loss during outages
Reactive ITSM	Tickets filed only after production impact; context lost at each handoff	Slow resolution, customer frustration
Talent constraints	Senior engineers as escalation gatekeepers	SLA pressure, retention risk
Post-migration drift	No continuous optimization or enforcement mechanism	Cloud waste of 30–40% of spend

Table 1: Current Support Model — Pain Domain Summary

Sources: Flexera 2026 State of the Cloud Report

2. The AaaS Paradigm Shift

The convergence of three forces makes AaaS both technically feasible and commercially urgent in 2026: the maturity of Gemini's reasoning and tool-use capabilities on Vertex AI; the standardization of agentic frameworks — Vertex AI Agent Builder, the Agent Development Kit (ADK), and the open Agent2Agent (A2A) protocol; and the widespread availability of Google Cloud-native telemetry — Cloud Logging, Cloud Monitoring, and Pub/Sub — that gives agents rich, real-time environmental context.

2.1 From Support Tiers to a Service Bouquet

AaaS replaces the rigid tier model with a composable marketplace of agents, delivered through Vertex AI Agent Builder and surfaced to customers via Agent Space. Customers browse a catalog of discrete agents — each encapsulating a specific domain of Google Cloud expertise — and subscribe only to what matches their current needs, priced per workload-hour of active engagement rather than per revenue band.

The Bouquet Model in Practice

Single-Project Application Scope: A project running a containerized microservice stack subscribes to: Autonomous SRE Agent + FinOps Controller + Project Security Sentinel Monthly spend is structured on a Flat Protected-Workload Base Fee + Outcome-Driven Resolution Units (measured against Mean Time to Autonomous Recovery [MTTR-A] benchmarks), directly aligning commercial cost with rapid resolution rather than billing for prolonged active-troubleshooting duration.

Tripartite Shared-Liability & SLA Framework: Each subscribed agent operates under an explicit tripartite liability model:

- **Platform Availability SLA (Google Cloud Responsibility):** Guarantees sub-minute perception and 99.9% platform availability on Vertex AI and Cloud Monitoring, backed by standard Google Cloud service credits.

- **Logic & Execution Liability (Cognizant / SI Responsibility):** The SI maintains primary liability for custom prompt definitions, Agent Development Kit (ADK) harnesses, and OPA/Rego policy templates. Direct operational downtime caused by agent logic defects or state-lock failures is indemnified up to $1,000,000 per incident or an agreed contractual cap via an Autonomous Operation Loss-Protection Rider.

- **Policy & Environmental Governance (Customer Responsibility):** The enterprise retains liability for infrastructure failures resulting from out-of-band manual console interventions, unmonitored external upstream APIs, or customer-authored OPA rule overrides.

SMB, e-commerce:

Migration Planner + Cost Optimizer + Security Baseline Agent

— monthly spend aligned precisely to three needs, with no wasted premium-tier coverage.

Enterprise, regulated industry (e.g., healthcare or financial services):

Full AaaS stack — ***Compliance Guardian, Incident Response, Architecture Advisor & FinOps Controller*** — with per-agent SLAs and measurable outcomes across the Google Cloud estate.

2.2 From Reactive to Predictive Operations

Agents operate in a continuous perception-reasoning-action loop built on Gemini models served through Vertex AI: ingest telemetry, reason over the customer's environment and history, and act — or recommend an action for approval — before a human would even notice a problem. This shifts the operating philosophy from reactive incident management to proactive, goal-driven autonomy, and shifts the primary success metric from response-time SLAs to resolution speed and zero-touch remediation rate.

2.3 Minimal Human Intervention by Design

AaaS does not eliminate humans from the loop; it relocates them. Under legacy support, humans are operators who execute every fix by hand. Under AaaS, humans become policy-makers who define the guardrails, review the exceptions an agent escalates, and approve high-risk actions — while agents handle everything within an approved policy envelope autonomously.

Dimension	Legacy Support Model	Agent as a Service
Operational philosophy	Reactive incident management	Proactive, goal-driven autonomy
Success metric	Response-time SLA	Resolution speed, zero-touch rate
Workload assignment	Manual triage, human queues	Autonomous agent routing and self-healing
Consumption model	Rigid, tiered packages	Modular, usage-based capability bouquet
Human role	Operator — executes fixes	Policy-maker — sets guardrails, approves exceptions

Table 2: Legacy Support vs. Agent as a Service

2.4 The Hyperscaler Landscape : A Narrowing Window

Google Cloud is not the only hyperscaler moving in this direction. Agentic capabilities are rapidly becoming an industry standard rather than a goal unique to any single provider.

AWS has introduced standalone "frontier agents" powered by Amazon Bedrock Agent Core, such as their DevOps Agent for autonomous incident investigation and a FinOps Agent for cost anomaly triage. Recently, AWS accelerated this push by announcing the public preview of the AWS Well-Architected Agent, an AI-powered service that analyzes cloud environments against best practices and delivers contextual recommendations with ready-to-implement fixes, including broad architectural patterns with the necessary Infrastructure as Code (IaC) changes.

Microsoft is following a similar trajectory with Azure Copilot's "agentic cloud operations," featuring a specialized Azure FinOps Agent alongside governance tools like Agent 365 and Foundry Agent Service.

Google Cloud already possesses highly equivalent technological capabilities today through Gemini Cloud Assist. For example, Gemini Cloud Assist allows users to design, configure, and manage complex cloud architectures using natural language, iteratively generate or refine Terraform code and `kubectl` manifests, and act as a user-guided site reliability engineer (SRE) to diagnose root causes across multi-tier topologies.

However, the critical gap in the market is not technological; it is commercial. As of now, no competitor has successfully integrated these features into a unified, consumption-based service model intended to *replace* traditional tiered support. Currently, AWS and Azure have only implemented separate point agents atop their existing commercial frameworks, requiring customers to maintain an expensive legacy support tier while adopting these agents independently. While the underlying orchestration frameworks, reasoning engines, and telemetry pipelines are becoming standard infrastructure across all major providers, the accompanying business models have yet to evolve.

This strategic opening represents a critical window of opportunity. While competitors offer disconnected, "black box" point-agents, Google Cloud's native advantage lies in its unified, extensible agentic stack—anchored by the **Agent Development Kit (ADK)** and Vertex AI.

Unlike rigid vendor-locked solutions, the Google ADK is an open-source, code-first framework explicitly designed for building modular multi-agent systems. Through its native Task API and the open Agent-to-Agent (A2A) protocol, ADK allows disparate agents to delegate tasks and collaborate seamlessly. Furthermore, ADK's graph-based execution engine natively supports deterministic routing, state management, and explicit Human-in-the-Loop (HITL) safeguards.

This means Google Cloud does not have to build a monolithic mega-agent; it can pioneer commercial disruption by delivering a coordinated "capability bouquet" where highly specialized ADK agents collaborate safely under enterprise-defined guardrails. The hyperscaler that successfully wraps these native multi-agent orchestration capabilities into an outcome-based, autonomous service model first will redefine cloud operations for the next decade.

2.5 Hybrid Topology & Telemetry Economics

Enterprise estates span hybrid interconnects, third-party edge appliances, and cross-cloud environments. AaaS avoids telemetry ingestion bloat by incorporating an **Edge Heuristic Pre-Filtering Gateway**:

Token-Efficient Ingestion: Suppresses high-volume routine telemetry at the edge, ingesting only scored anomalies and delta states into Vertex AI Vector Search to preserve 65–80% gross margins against inference overhead.

Hybrid Adapter Connectors: Uses standard OpenTelemetry collectors to consume telemetry and dispatch remediation actions across Google Cloud, Anthos, and cross-cloud interconnects without requiring homogenous GCP infrastructure.

3. The AaaS Agent Portfolio

Rather than a single monolithic assistant, AaaS is delivered as a portfolio of specialized agents, each scoped to a domain of Google Cloud expertise and orchestrated via the Agent2Agent (A2A) protocol. The portfolio is organized into four pillars that map to the customer's journey from initial adoption through continuous operation.

Scope Governance (Platform Fabric vs. Application Space):

AaaS operates with deterministic scope boundaries:

1. **Core Platform Support (Autonomous / Stage 3):** Single-project infrastructure remediation (GKE node health, Cloud SQL auto-tuning, project-level IAM reconciliation and service-project subnet route telemetry).
2. **Cross-Project & Shared Topology Boundary (Federated Inspection & Stage 2 HITL Routing):** In enterprise hub-and-spoke or Shared VPC topologies, the agent is granted cross-project Read-Only Network Topology & Telemetry Inspection rights to rule out transit VPC packet drops or host-project route tables. Any mutating operation targeting Shared VPC Host Projects, Cross-Project Peering, or Centralized Service Perimeters is strictly prohibited from Stage 3 autonomy and automatically routes to a Stage 2 HITL federated approval gate.

3. **Application-Level & Migration Operations (Assisted / Stage 2 HITL & Design Accelerator):** Migration wave planning, landing zone code generation, schema transformations, and application code optimizations operate strictly as Design-Time Accelerators and Advisory PR Generators requiring human approval.

3.1 Migration & Adoption Agents

- **Architectural Design & Suggestion Agent:** An AI agent that can help customer with comparing their current state architecture and generate a Target state architecture, Including TCO Analysis and right sizing insights
- **Cloud Migration Navigator Agent:** performs portfolio assessment, wave planning, and TCO modeling, integrating with Migration Center to sequence a workload migration.
- **Landing Zone / Infrastructure Provisioning Agent:** Generates and applies Infrastructure as Code for multi-project folder structures, VPC networking, and IAM foundations, and remediates configuration drift automatically.
- **Data Migration & Validation Agent:** handles schema transformation, incremental sync, and integrity validation using Database Migration Service and Datastream.

3.2 Operations & Reliability Agents

- **Autonomous Incident Response (SRE) Agent:** detects anomalies, executes auto-remediation playbooks, and produces an escalation brief when a novel failure pattern exhausts its playbook library.
- **Capacity & Performance Optimization Agent:** right-sizes resources and adjusts autoscaling policy ahead of demand, using Active Assist and the Recommender API.
- **Chaos & Validation Agent:** runs controlled failure-injection experiments to surface single points of failure and confirm that remediation logic behaves as intended before it is trusted with production incidents.

3.3 Security & Compliance Agents

- **Security Sentinel Agent:** provides continuous cloud security posture management and threat detection via Security Command Center and Chronicle, auto-remediating routine findings and escalating confirmed threats with legal or regulatory implications.
- **Compliance Guardian Agent:** maps controls to applicable frameworks (for example HIPAA, PCI-DSS, or ISO 27001), collects evidence continuously, and scores posture using Security

Command Center and Assured Workloads — replacing point-in-time audits with continuous enforcement.

3.4 Financial & Advisory Agents

- **FinOps Controller Agent:** optimizes spend, manages commitment strategy, and flags anomalies using BigQuery billing export and Active Assist.
- **Architecture & Maturity Advisor Agent:** reviews workloads against the Google Cloud Architecture Framework, identifies modernization opportunities, and tracks maturity using adoption-framework benchmarks and DORA metrics.

This is a representative slice of the catalog, not an exhaustive one — the model is designed so that new agents can be added to any pillar as customer needs and Google Cloud's own product surface evolve. A fuller capability reference appears in Appendix A.

4. The Autonomous Workload Lifecycle

Each agent interaction follows the same four-stage life cycle, with a defined escalation path back to a human whenever confidence or policy requires it.

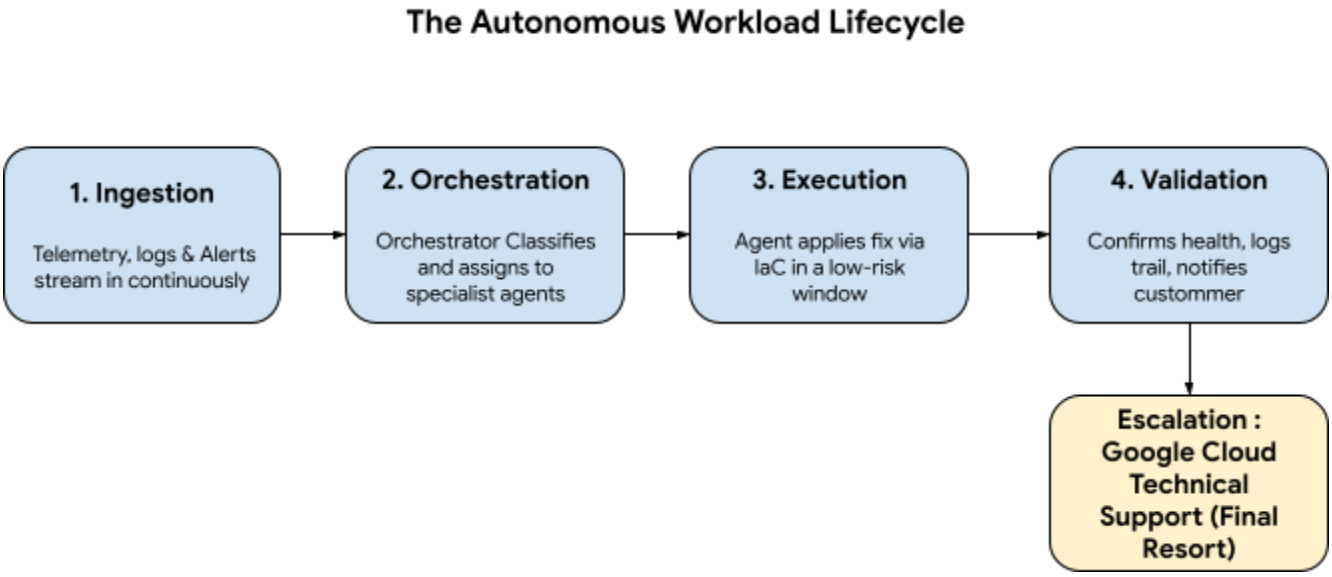


Figure 1: The Autonomous Workload Lifecycle, with escalation to human specialists as the final resort

1. Ingestion: the environment continuously streams telemetry — Cloud Logging events, metrics, and security alerts — into the AaaS platform.

2. Orchestration: an orchestrator agent classifies the signal, breaks it into sub-tasks, and assigns them to the specialized agent(s) best suited to act — for example, looping in the FinOps Controller to confirm budget clearance before the SRE agent expands a disk volume.

3. Execution, Concurrency Locking & Blast Radius Containment: Before applying mutations, the agent executes an external dependency check to confirm that anomalies are not caused by third-party APIs or cross-project services. It then executes a three-point concurrency lock: it acquires an exclusive Terraform remote state lock (gs://<tf-state-bucket>/<env>.tflock) initialized with a 10-minute lease TTL. During Stage 2 HITL review, the Agent Harness runs an active Lease-Keeper Heartbeat Daemon via the operator interface that dynamically renews the GCS lock lease in 5-minute in\crements as long as an active engineer review session is open (auto-evicting only upon 10 minutes of complete operator inactivity). Concurrently, it signals GitOps reconcilers (e.g., ArgoCD/Flux) to temporarily pause target application sync loops with an automated fail-safe resume timer and sets a distributed mutex across active CI/CD webhooks. Ephemeral credentials minted via Workload Identity Federation are restricted to single-action execution envelopes with mid-flight session-revocation hooks. It captures a declarative resource snapshot, verifies baseline metrics, and progresses canary execution in lockstep with automated reconciliation PRs to prevent drift regressions.

4. Telemetry Watchdog, Fail-Closed Dead-Man's Switch & Auto-Rollback: Following ,execution, the platform monitors telemetry health. A synthetic heartbeat metric (custom.googleapis.com/aaas/heartbeat_tick) is verified every 15 seconds; if telemetry latency exceeds 45 seconds, the agent **fails closed**, halts write operations, cleanly releases the GCS lease lock, and reverts to HITL mode. If telemetry indicates system degradation, the auto-rollback engine atomically reverts to the pre-execution

snapshot, generates an automated reconciliation PR for the Git repository, and routes an escalation brief to on-call engineers.

Bifurcated Stateful vs. Stateless Auto-Rollback Engine: Rollback actions are deterministically bifurcated based on resource persistence:

- **Stateless Compute (GKE Pods, Cloud Run, MIGs):** Automated, atomic canary rollback to the prior versioned deployment spec or container image digest.
- **Stateful Storage & Data (Cloud SQL, Spanner, GCS, Persistent Disks):** The agent strictly avoids destructive out-of-band rollbacks. Instead, it executes an automated **Safe-Mode Isolation**: capturing an instant point-in-time snapshot, engaging synthetic upstream circuit breakers to shed non-essential connection pools and avoid thread exhaustion, isolating suspect transactions, and escalating to a Stage 2 HITL specialist with pre-compiled diagnostic traces.

If an agent exhausts its playbook — a truly novel failure pattern, a policy conflict, or a threat with legal implications — **it escalates to the "Google Cloud technical support team"** as the final resort, handing over a complete summary of every action already attempted so a human specialist starts from full context rather than zero.

5. Industry-Agnostic Approach

To maximize the value of the Agent as a Service (AaaS) platform, the architecture and product strategy have been designed to be fundamentally industry-agnostic. Rather than hardcoding the platform to resolve specific IT or engineering incidents (e.g., GKE errors), the system abstracts the core capabilities—orchestration, knowledge retrieval, human-in-the-loop (HITL) approvals, and execution—so they can be applied across any domain.

• **Domain-Agnostic Orchestration:** The Vertex AI Orchestrator utilizes a generic ReAct loop that can ingest domain-specific schemas, APIs, and instructions dynamically.

• **Flexible Knowledge Base:** The Vertex AI Vector Search can be populated with IT playbooks, financial compliance guidelines, healthcare protocols, or customer service manuals.

• **Adaptable Execution Guardrails:** The Execution Sandbox enforces policies based on the imported domain context, ensuring that actions taken by the agent remain compliant regardless of the industry.

6. Go-to-Market & Expansion Plan

To effectively scale the Agent as a Service (AaaS) platform and cater to varying levels of enterprise cloud maturity, we will employ a dual-track GTM strategy:

6.1 Account Team-Led Deployment

Enable the AaaS platform directly through Google Cloud Account Teams and Customer Engineers (CEs). In this high-touch model, the account teams act as our "eyes and ears" on the ground. By deeply understanding the customer's specific architectural pain points, they can identify the highest-value use cases and provide white-glove assistance to deploy the agent portfolio directly into customer projects. This hands-on approach ensures a guided progression through the Adoption Maturity Model, helping clients safely structure their Human-in-the-Loop (HITL) approval gates and transition smoothly to autonomous operations.

6.2 Google Cloud Marketplace (Self-Service)

Enable AaaS as a packaged, self-deployable solution within the Google Cloud Marketplace. Designed for digitally native or highly mature engineering organizations, this option provides comprehensive installation guides and pre-configured Terraform/OpenTofu modules. It empowers customers to install the Agent Harness, the Execution Sandbox, and their selected "capability bouquet" autonomously onto their own environments. This frictionless channel accelerates broad market adoption and allows customers to start experimenting with Stage 2 (HITL) auto-remediation immediately, with zero mandatory consulting overhead.

6.3 Resilient and Enterprise Standard

The foundation of the platform relies on hardened security and highly resilient AI orchestration. By containerizing the tool control layer within a secure Execution Sandbox, we prevent unauthorized actions, contain the blast radius of any automated task, and mitigate hallucination risks. Deep integration with Vertex AI ensures the platform meets strict enterprise standards, providing the guaranteed uptime, privacy compliance, and low-latency performance necessary for mission-critical production deployments.

6.4 Ease of Implementation

Adoption friction is minimized through the "Agent Harness Wrapper," which allows clients to connect their existing tools and environments (e.g., GCP Logging, external APIs) without extensive re-architecture. The inclusion of an out-of-the-box Active Web UI Dashboard provides instant visibility and control, requiring zero frontend development from the client to establish a Stage 2 HITL review process.

6.5 Commercial Expansion

We will employ a "land and expand" strategy. The initial landing will focus on high-pain, easily quantifiable areas like IT Operations (as demonstrated in the PoC) and DevOps auto-remediation. Once trust is established via the HITL approval loops, clients can expand the AaaS platform into

customer support automation, financial transaction auditing, and business process automation, unlocking modular licensing opportunities for each new domain and integration.

6.1 Technical Solidity

The foundation of the platform relies on hardened security and reliable AI orchestration. By containerizing the tool control layer within a secure Execution Sandbox, we prevent unauthorized actions and mitigate hallucination risks. The integration with Vertex AI provides enterprise-grade uptime, privacy compliance, and latency guarantees necessary for production deployments.

6.2 Ease of Implementation

Adoption friction is minimized through the "Agent Harness Wrapper," which allows clients to connect their existing tools and environments (e.g., GCP Logging, external APIs) without extensive re-architecture. The inclusion of an out-of-the-box Active Web UI Dashboard provides instant visibility and control, requiring zero frontend development from the client to establish a Stage 2 HITL review process.

6.3 Commercial Expansion

We will employ a "land and expand" strategy. The initial landing will focus on high-pain, easily quantifiable areas like IT Operations (as demonstrated in the PoC) and DevOps auto-remediation. Once trust is established via the HITL approval loops, clients can expand the AaaS platform into customer support automation, financial transaction auditing, and business process automation, unlocking modular licensing opportunities for each new domain and integration.

7. Financial Business Case

The ROI of AaaS has two distinct beneficiaries: the organization that delivers the model, and the enterprise customer that consumes it.

AaaS shifts the provider from a labor-bound managed-services business to a higher-margin, software-like one. Industry benchmarking of managed service providers puts typical net margins at 20–30% (roughly 50–60% gross margin), constrained by billable-hour economics and headcount growth. Software and consumption-priced platforms, by contrast, typically report 75–85% gross margins once the delivery mechanism is largely automated. As the Vertex AI orchestration layer and agent blueprints mature, AaaS moves the provider's cost structure from the first category toward the second, decoupling revenue growth from headcount growth. Sticky, outcome-based agent subscriptions also tend to raise customer lifetime value relative to a swappable annual support contract.

At the single-project level, edge heuristic filtering suppresses >90% of routine telemetry before LLM reasoning. An average of ~150 triage invocations/month per project incurs <$2.00/month in Gemini token inference costs. Combined with shared runtime overhead (~$200/month per project), the provider preserves a 65–80% gross margin on a $1,000/month subscription.

Sources: MSP net/gross margin benchmarks — V2 Cloud • MSP margin benchmarks — Bering McKinley • SaaS gross margin benchmarks — Aleph × Benchmarkit 2026

Frequently cited risk-avoidance benefit for regulated industries is exposure to compliance penalties. Under current HHS Office for Civil Rights rules, the most severe ("willful neglect, uncorrected") tier of HIPAA civil penalty carries a per-violation and annual cap of roughly $2.1–$2.2 million — continuous, automated compliance monitoring is aimed at avoiding exposure at that tier rather than eliminating audit cost outright.

Sources: HHS 2026 HIPAA civil penalty adjustment • HIPAA Journal — 2026 penalty tiers

On the operating-cost side of the ledger, published AI-agent automation benchmarks — largely drawn from customer-support and IT-service-desk deployments rather than cloud infrastructure specifically — report cost reductions in the 40–60% range for eligible ticket categories, with narrower, highly repetitive categories seeing 80%+ reductions, and Gartner projecting industry-wide contact-center savings of roughly $80 billion by the end of 2026. These figures are the closest available external anchor for the Tier 1/2 savings AaaS targets; no published study isolates cloud-infrastructure incident response specifically, so the "up to 70%" figure used elsewhere in this paper should be read as directionally consistent with, rather than proven by, this data.

Sources: Gartner-sourced ROI benchmarks — Fin.ai • Cost-per-ticket automation benchmarks — Lorikeet • AI ticket-resolution ROI benchmarks

8. Technical Architecture Blueprint

AaaS is built as a layered stack that translates raw telemetry into safe, autonomous action, with a guardrail layer that gates every layer capable of touching a production system.



8.1 Current Model vs. Proposed AaaS model (Agent Triad)

To view an interactive comparison between the traditional GCP support architecture and the proposed multi-agent automated framework, choose "Agent Triad"—located as the final icon on the menu row.

Figure 2: AaaS Technical Architecture — layered stack with an independent guardrail layer spanning every layer

Layer	Function	Representative Google Cloud Services
Telemetry ingestion & Filtering	Edge anomaly pre-filtering with a 15s synthetic heartbeat dead-man's switch	Cloud Logging/Monitoring, Pub/Sub, Cloud Functions (Heartbeat Generator)
Context & memory fabric	Maintains durable state history, cross-project dependency topology graphs (Cloud Trace/Network Topology), and causal baselines with token-capped vector retrieval	Vertex AI Vector Search, BigQuery, Spanner Graph, Cloud Trace
Multi-Agent Orchestration & Arbitration	Decouples intent, manages A2A protocol routing, and enforces DAG-based cyclic conflict arbitration with a 45-minute temporal hysteresis freeze	Vertex AI Agent Builder, A2A Protocol, Directed Acyclic Graph (DAG) Arbitration Engine, Temporal Hysteresis Controller
Agent Harness & Execution Sandbox	Intercepts model tool calls, acquires Terraform GCS state locks, manages JIT tokens, pauses GitOps loops, and enforces speculative CI/CD dry-run validation on generated PRs	Agent Runtime Harness, Cloud Build CLI Runner, Config Connector, Terraform/OpenTofu Reconciler, Cloud Deploy
Security, JIT & Policy Guardrails	Enforces zero-standing privileges,token-bucket rate limiter, OPA/Rego policies, Model Armor, and destructive action blocks	Workload Identity Federation, JIT Access Broker, Cloud KMS, OPA/Rego, custom python rate limiter
Blast Radius, Semantic Probing & Auto-Rollback	Canary staged deployments, snapshot checkpointing, post-execution active semantic probing (canary traffic routing) and telemetry watchdog rollbacks	Security Command Center, Assured Workloads, Cloud Monitoring, Cloud Trace, Synthetic Monitoring Probers

Table 4: AaaS Technical Architecture Layers

Note : *To ensure absolute safety, the Guardrail Layer operates on a zero-trust model. High-risk actions (e.g., identity access modifications, production network changes) are strictly gated behind*

Human-in-the-Loop (HITL) approval. Furthermore, the platform utilizes isolated sandboxes to dry-run and validate Infrastructure-as-Code (IaC) changes before they touch production.

Customer intent enters the stack through a natural-language interface — "optimize this GKE cluster for cost" rather than a technical parameter form — and the orchestrator decomposes that intent into sub-tasks for the appropriate specialist agents. A deterministic guardrail layer sits alongside every layer that can act on the customer's environment: no agent output reaches production without passing an independent, rules-based policy check, and any action affecting a regulated data boundary (for example, protected health information) additionally requires the escalation path described in Section 8.

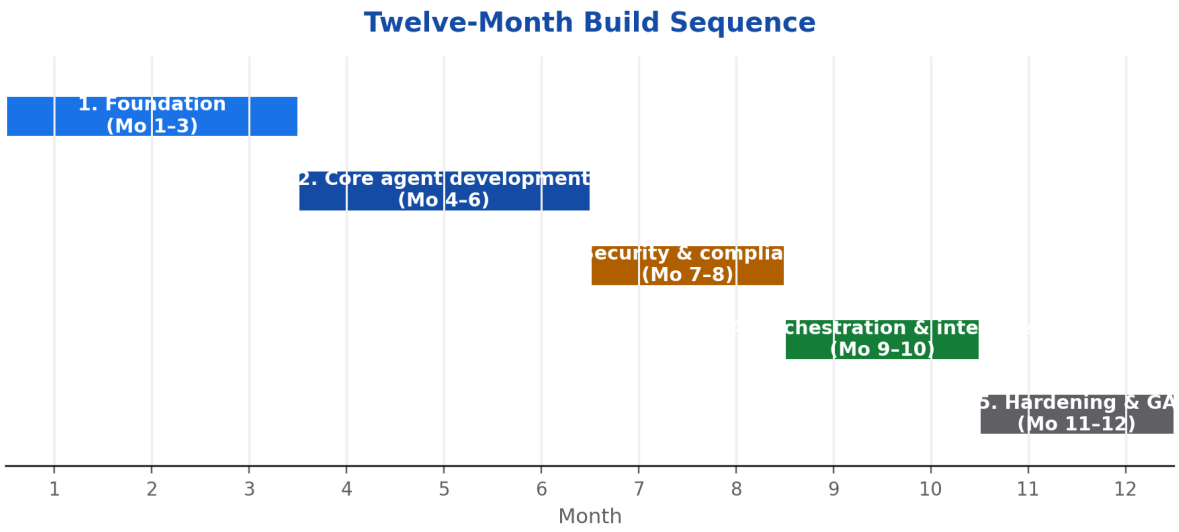
9. Implementation Roadmap

9.1 Phased Build

A twelve-month build sequences platform foundations before autonomy, and reliability agents before advisory ones, so that trust and technical scaffolding are established before the highest-leverage agents go live.

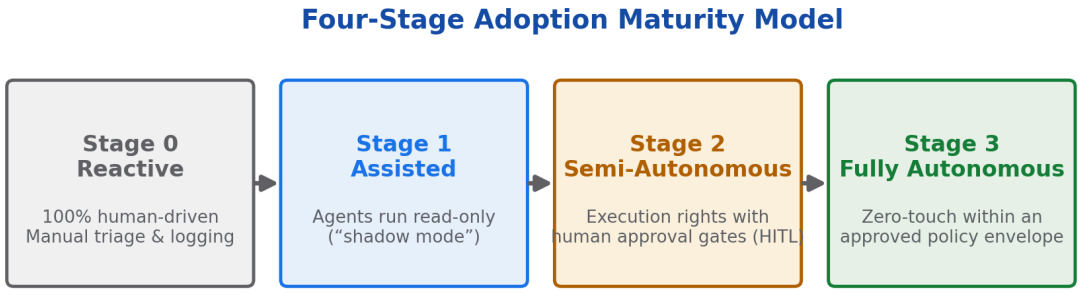
Phase	Timeline	Focus
1. Foundation	Months 1–3	Telemetry pipeline, memory infrastructure, and core platform setup
2. Core agent development	Months 4–6	Incident Response, Migration Navigator, and FinOps Controller agents
3. Security & compliance	Months 7–8	Security Sentinel and Compliance Guardian agents
4. Orchestration & interface	Months 9–10	Multi-agent orchestrator and customer-facing interface layer
5. Hardening & GA	Months 11–12	Security hardening, load testing, and automated customer onboarding

Table 5: Twelve-Month Build Sequence



9.2 Adoption Maturity Model

Moving an existing customer relationship from pure human operation to full autonomy should not be a single cutover — particularly in regulated environments. A four-stage maturity model governs the transition, with each stage graduating only once the prior stage's accuracy and safety metrics are met.



Progression is graduated: each stage advances only once accuracy and safety metrics for the prior stage are met.

Stage	Operational Mode	Primary Metric
0 — Reactive	100% human-driven; manual ticket logging, triage, and assignment	Response-time SLA (e.g., P1 acknowledged in 15 minutes)

Stage	Operational Mode	Primary Metric
1 — Assisted (shadow)	Agents run read-only, drafting recommended fixes without executing them	AI recommendation acceptance rate
2 — Semi-autonomous (HITL)	Agents have execution rights but pause at conditional approval gates for higher-risk changes	Time-to-resolution; reduction in manual labor hours
3 — Fully autonomous	Zero-touch operation within an approved policy envelope; humans set guardrails, not execute fixes	MTTR, zero-touch resolution rate, compliance posture score

Table 6: Four-Stage Adoption Maturity Model

Graduation Protocol (Multivariate Verification Gate): Transitioning a workload or remediation playbook from Stage 2 (HITL) to Stage 3 (Fully Autonomous) requires satisfying three concurrent criteria:

(1) A minimum of 30 successful, human-verified remediations with a ≥99.0% recommendation acceptance rate;

(2) Operational validation across at least two distinct environmental edge states (e.g., peak-traffic load windows, maintenance windows, or simulated upstream latency)

(3) Zero guardrail violations or unmanaged PR merge conflicts during the qualification period.

Action-Type Tiered Autonomy: Graduation is strictly partitioned by operational blast radius:

Autonomous Candidates (Stage 3): Safe, fully idempotent actions (e.g., GKE pod vertical/horizontal scaling, disk volume extensions, non-destructive cache flushing) graduate to unsupervised autonomy once thresholds are satisfied.

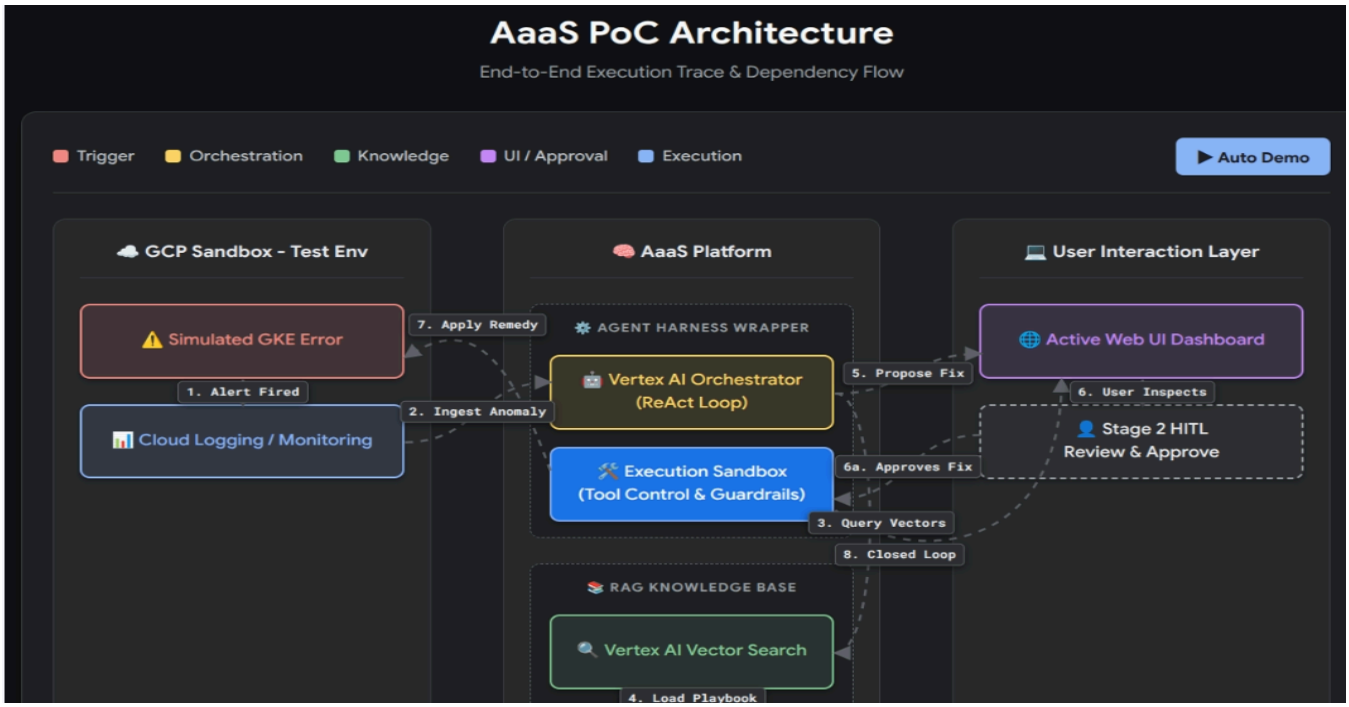
Permanently Supervised Actions (Stage 2 HITL): Non-idempotent or high-blast-radius operations (e.g., project IAM role modifications, subnet route alterations, persistent disk snapshot purges, stateful database reboots) remain permanently gated behind human approval.

Approval-Fatigue Telemetry Guard: To prevent rubber-stamping, the Agent Harness logs human operator interaction metrics (e.g., time-on-diff inspection, parameter adjustments) before marking a dry-run approval as valid training signal.

Scope Boundary Enforcement: Any remediation requiring Org-level or Folder-level intervention bypasses autonomous graduation entirely and immediately routes to the Google Cloud technical support team or customer cloud operations.

10. AaaS PoC Architecture

The AaaS Proof of Concept demonstrates an end-to-end execution trace and dependency flow, broken down into three core environments: the GCP Sandbox (Test Environment), the central AaaS Platform, and the User Interaction Layer.



Execution Trace & Dependency Flow

1. Alert Fired: A simulated anomaly (e.g., a GKE Error) is triggered within the GCP Sandbox environment.

2. Ingest Anomaly: Cloud Logging/Monitoring captures the event and routes it to the AaaS Platform, specifically into the Vertex AI Orchestrator running a ReAct (Reasoning and Acting) loop.

3 & 4. Query Vectors & Load Playbook: The Orchestrator queries the RAG Knowledge Base via Vertex AI Vector Search to retrieve the appropriate runbooks or resolution strategies based on the anomaly.

5. Propose Fix: Once a remediation strategy is formulated, the Orchestrator pushes a proposed fix to the User Interaction Layer.

6. User Inspects & Approves (HITL): Through the Active Web UI Dashboard, a human operator performs a Stage 2 Review. Upon inspection, the user approves the fix (Step 6a), authorizing the action.

7. Apply Remedy: The authorized command is routed to the Execution Sandbox, which enforces strict tool control and guardrails, before applying the remedy back to the GCP Sandbox to resolve the simulated error.

8. Closed Loop: The Execution Sandbox reports the status of the remedy back to the Orchestrator, closing the automation loop and updating the monitoring state.

11. Governance, Risk, and Responsible AI

Google Cloud

Autonomy is only durable if it is trustworthy. AaaS is built around an explicit guardrail framework rather than relying on model judgment alone:

- **Deterministic policy gates, network blast-radius containment & state parity:** Every agent-generated action passes through independent policy checks (e.g., OPA/Rego) and speculative GitOps dry-runs before execution. Autonomous L1 single-project agents are hard-restricted from modifying Shared VPC host project routes, interconnects, or cross-project peering boundaries (routing any such intent to Stage 2 HITL approval). Agent-driven modifications enforce source repository synchronization to maintain state locks and eliminate unmanaged configuration drift across enterprise CI/CD workflows.
- **Pipeline Concurrency Locking & Deterministic GitOps Reconciliation:** Agents programmatically lock remote Terraform GCS backends and pause CI/CD webhooks during active remediations. Following an applied hotfix, the agent submits an automated reconciliation PR accompanied by speculative dry-run lint and policy checks. If the generated PR fails CI/CD integration tests within a 15-minute window, the agent flags an unmanaged drift alert, triggers safe-mode isolation on the modified resource, and routes an escalation brief to on-call engineering to prevent persistent repository-to-state drift.
- **Fail-Closed Circuit Breaker (Dead-Man's Switch):** If telemetry pipeline latency exceeds 45 seconds, all automated write permissions are suspended, and the agent drops to Shadow/HITL mode.
- **API Quota Preservation & Hourly Resource Blast-Radius Governor:** Hardcoded rate limits restrict agent mutations to <=30% of project API quotas. Furthermore, autonomous Stage 3 execution is bounded by an hourly resource envelope: an agent cannot modify more than 2 distinct workloads or alter more than 20% of aggregate compute capacity within any rolling 60-minute window without escalating to a Stage 2 HITL approval gate.
- **Tiered approval:** low-risk, well-understood actions (non-production scaling, routine patching) execute autonomously; high-risk actions (encryption key changes, movement of regulated data) require explicit human approval regardless of the agent's confidence.
- **Immutable audit trail:** every action, script, and validation step is logged to the ticketing system and retained for the compliance period the customer's industry requires, replacing manual documentation with a continuous, tamper-evident record.
- **Responsible AI alignment:** Agent design follows Google's Responsible AI principles and Secure AI Framework (SAIF). To protect against Indirect Prompt Injection (IPI) within runtime logs and stack traces, an intermediate telemetry sanitization engine extracts typed JSON schemas before context ingestion. Furthermore, Model Armor screens all conversational and tool-call I/O boundaries.
- **Deterministic Conflict Arbitration & Anti-Flapping:** Multi-agent operational intents resolve via a strict hierarchy:

  Reliability & Availability > FinOps Cost > Modernization / Advisory.

  To eliminate sequential resource thrashing (flapping loops), the platform enforces a **Temporal Hysteresis Engine** with a mandatory 45-minute parameter freeze on any resource envelope

modified by an active SRE remediation before FinOps right-sizing can re-engage. Lower-priority agent intents are paused with a conflict explanation logged to the audit trail.

- **Zero-Standing Privilege (JIT Broker):** Agents operate without persistent write credentials. Upon validation of an approved execution plan, an ephemeral service account token is minted via **Workload Identity Federation** with short-lived TTLs (e.g., 15 minutes) and cryptographically signed audit logs tied to the incident ID.
- **Deterministic Auto-Rollback & Dynamic Poison-Pill Quarantine:** All mutating operations require pre-execution state snapshots and canary boundaries. If real-time monitoring detects anomalous telemetry degradation within the post-change watchdog window, the agent automatically executes an atomic rollback to the prior known-good state, releases the GCS lease lock, and logs the execution signature to a dynamic **Poison-Pill Quarantine Registry**. Any quarantined remediation path is locked out from autonomous re-execution on that resource for 7 days, immediately falling back to Stage 2 HITL escalation upon recurrence.
- **Out-of-Band Mutation Detection & Transactional Safe-Yield Engine:** The platform monitors Cloud Audit Logs via Pub/Sub for manual user-initiated mutations (e.g., gcloud, GCP Console, or direct API modifications). The moment a manual write action is detected on an active resource, the Agent Harness executes a two-phase safe-yield: it completes or atomically rolls back the immediate in-flight step, cleanly releases the GCS Terraform remote state lock, emits a handoff summary to the incident audit trail, and switches the agent to passive "Observation Mode" until human on-call engineers explicitly clear the lock and re-engage autonomy.

Change management matters as much as technology. Customers should expect a deliberate transition of their own teams from operators executing tickets to policy owners who curate the guardrails an agent operates within — a role change that benefits from the phased maturity model in Section 7.2 rather than a single go-live event.

12. Conclusion & Strategic Advantage

The traditional support model — tiered SLAs, reactive ticketing, and labor-intensive consulting — was not built for the pace and complexity of modern cloud transformation. It creates friction at every point in the customer journey and fails to leverage what Gemini and Vertex AI can already do today.

Agent as a Service reimagines that model for Google Cloud. By putting purpose-built agents — built on Vertex AI Agent Builder and orchestrated via the Agent2Agent protocol — at the center of the customer journey as autonomous operators and always-on advisors, AaaS aligns Google Cloud and its customers around shared, measurable outcomes: customers pay for what they use and receive

continuous, compounding value, while Google Cloud builds a scalable business that gets more capable with every customer interaction.

The Imperative

The question for Google Cloud is not whether to build Agent as a Service capabilities, but how quickly.

Early movers establish the data network effects, playbook libraries, and customer trust that become durable competitive moats — while slower-moving organizations risk falling behind in a market that is repricing cloud expertise from human labor hours to autonomous agents running on Vertex AI.

This white paper has laid out a practical blueprint: the pain points of the current model, the strategic rationale for AaaS, a representative agent portfolio and workload lifecycle, the financial case for both provider and customer, a technical architecture built on Google Cloud, and a phased roadmap with an adoption maturity model and governance framework. The path from today's state to AaaS is substantial but navigable — and the organizations that navigate it first will define the next decade of cloud services delivery.

Appendix A: Agent Capability Quick Reference

Agent	Pillar	Primary Capability	Escalation Trigger
Migration Navigator	Migration & Adoption	Portfolio assessment, wave planning, TCO modeling	Novel dependency pattern; exec alignment needed
Provisioning Agent	Migration & Adoption	IaC generation, drift remediation	IaC apply failure after repeated retries
Data Migration Agent	Migration & Adoption	Schema transform, incremental sync, validation	Data integrity violation beyond threshold
Incident Response (SRE) Agent	Operations & Reliability	Anomaly detection, auto-remediation	Playbook exhausted; novel failure pattern
Capacity Optimizer	Operations & Reliability	Right-sizing, autoscaling, performance analysis	Conflicting performance/cost signal
Security Sentinel	Security & Compliance	Posture management, threat detection, auto-remediation	Confirmed threat actor; legal/regulatory implication
Compliance Guardian	Security & Compliance	Framework mapping, evidence collection, posture scoring	New regulatory requirement outside current scope
FinOps Controller	Financial & Advisory	Spend optimization, commitment management	Customer disputes savings attribution

Agent	Pillar	Primary Capability	Escalation Trigger
Architecture / Maturity Advisor	Financial & Advisory	Framework review, modernization roadmap	Fundamental architecture refactor or C-suite alignment

Table A1: AaaS Agent Capability Quick Reference

Appendix B: Glossary

A2A (Agent2Agent Protocol): An open protocol for structured communication and task delegation between autonomous AI agents.

Agent Harness: The deterministic runtime layer and execution sandbox that encapsulates the LLM, managing tool dispatches, JIT credential lifecycles, state locking, policy evaluation (OPA/Rego), and auto-rollback mechanics.

AaaS: Agent as a Service — the delivery model described in this white paper.

ADK (Agent Development Kit): Google's open-source framework for building and orchestrating multi-agent systems on Vertex AI.

Agentic AI: AI systems capable of autonomous, multi-step reasoning and action execution.

Agentspace: Google Cloud's interface for discovering and interacting with enterprise AI agents.

FinOps: Financial Operations — the discipline of managing cloud financial accountability.

Gemini: Google's family of multimodal foundation models, accessed via Vertex AI.

IaC (Infrastructure as Code): Infrastructure defined and managed through versioned code artifacts.

ITSM: IT Service Management — the ticketing and workflow platform used for support operations.

MTTR: Mean Time to Resolution (or Remediation) — a key incident-management performance metric.

SAIF (Secure AI Framework): Google's framework for building and deploying AI systems securely.

Vertex AI: Google Cloud's unified platform for building, training, and deploying ML models and AI agents.

Bidirectional GitOps Reconciliation: An operational mechanism allowing autonomous agents to execute hot-patches on live infrastructure while concurrently synchronizing declarative repositories and remote state backends to prevent pipeline collisions and configuration drift.

Blast Radius Containment & Auto-Rollback Engine: An automated safety mechanism that bounds the impact of infrastructure modifications through canaries and snapshots, autonomously reverting systems to their prior state if latent telemetry regressions appear.

Appendix C: Consolidated References

All external sources cited throughout this paper, grouped by the section in which each is used. Figures not listed here are illustrative modeling assumptions built for this paper and are flagged as such in-line.

Section 1 — The Limits of Legacy Cloud Support

Flexera 2026 State of the Cloud Report

Flexera — "Cloud Value Is Rising While AI Waste Grows" (press release)

Section 5.1 — Benefits to the Service Provider

V2 Cloud — MSP Profit Margins benchmark

Bering McKinley — MSP profitability KPIs

MSP Success — "The Profitability Crisis in the Managed Services Industry"

Aleph × Benchmarkit — 2026 SaaS & AI Performance Benchmarks

Section 5.2 — Benefits to the Customer

Flexera 2026 State of the Cloud Report (cloud waste rate)

ITIC 2024 Hourly Cost of Downtime Survey, via Gatling

Ponemon Institute downtime-cost benchmark, via HDTech

HHS — 2026 HIPAA civil monetary penalty adjustment

HIPAA Journal — 2026 penalty tiers

Fin.ai — ROI of AI Customer Service benchmarks (Gartner-sourced)

Lorikeet — Cost-per-ticket automation benchmarks

Sections 2–4 and 6–9 describe the proposed operating model, architecture, and governance framework; the specific mechanisms, agent names, and workflow patterns are this paper's own design synthesis rather than claims requiring external citation.

INCIDENT RESOLUTION WORKFLOW COMPARISON